THE LAST SIGNAL

C++ Text-Based Console Adventure Game

Game concept • Technical requirements • Storyline • Architecture • Build guide

Language	C++20
Target platform	Windows 10 / Windows 11
Application format	Standalone console executable (.exe)
Interface	Keyboard input and text output
Recommended level	Beginner to intermediate

Project purpose
This document defines a practical first version of a C++ console adventure game. It explains what to build, how the game should behave, which libraries to use, how to organise the source code, and how to compile the project into a Windows executable.

Version 1.0 • 9 November 2026

1. Project Overview

The Last Signal is a single-player, text-based adventure RPG. The player arrives in a remote settlement after receiving a mysterious emergency transmission. The settlement appears abandoned, the surrounding forest is dangerous, and an underground research facility may contain the explanation.

The game is played entirely through text. The console displays descriptions, choices, status information and combat messages. The player types commands or selects numbered options.

Main gameplay features

- **Exploration:** visit locations, inspect surroundings and discover clues.
- **Story choices:** make decisions that affect characters, events and the ending.
- **Inventory:** collect, inspect and use items such as keys, medicine and tools.
- **Combat:** fight hostile creatures using health, damage and consumable items.
- **Puzzles:** find clues and enter codes to unlock areas.
- **Progression:** complete objectives and unlock new locations.
- **Save and load:** store the player's progress in a local save file (recommended feature).

Core gameplay loop

```
Start game
   |
Show introduction and player status
   |
Display current location and available actions
   |
Read and validate player input
   |
Perform action: explore / inspect / move / inventory / fight
   |
Update player, world and quest state
   |
Check death, victory or story progression
   |
Repeat until the player quits or reaches an ending
```

The game should remain playable without a mouse, graphics engine or internet connection.

2. Storyline and World

Premise

A remote settlement called Greyhaven has gone silent. Three nights ago, an emergency radio signal was transmitted from the settlement's old observatory. You play as Alex Rowan, a field technician sent to investigate. When you arrive, the road behind you collapses in a storm. Your radio receives one repeated message: “Do not let the signal reach the tower.”

The mystery

Years earlier, a research team beneath Greyhaven built a device designed to transmit messages across impossible distances. During an experiment, the device connected to something unknown. The research team disappeared, the settlement was evacuated, and the facility was sealed. Now the signal has started again.

Chapter outline

Chapter	Location	Main objective
1. The Arrival	Greyhaven outskirts	Reach the settlement and find a working radio.

Chapter	Location	Main objective
2. Empty Streets	Abandoned village	Search houses, collect supplies and locate the observatory key.
3. The Old Forest	Forest trail	Avoid or fight creatures and find the research access card.
4. Below Ground	Research facility	Solve the power and access puzzles; uncover the experiment.
5. The Last Signal	Signal tower	Choose whether to destroy, contain or activate the device.

Possible endings

- **Containment ending:** shut down the device and prevent the signal from spreading.
- **Transmission ending:** activate the device to send a warning, accepting the unknown consequences.
- **Destruction ending:** overload the facility and destroy the source, escaping before it collapses.
- **Bad ending:** run out of health or make a critical choice without the required item or information.

The ending can depend on quest flags, collected evidence, player choices and whether key items were found.

3. Functional Requirements

Functional requirements describe what the game must allow the player or system to do.

ID	Requirement	Priority
FR-01	Display a title screen with New Game, Load Game (if implemented) and Quit.	Must
FR-02	Allow the player to enter a name or use a default name.	Should
FR-03	Display the current location and its description.	Must
FR-04	Allow movement between unlocked locations.	Must
FR-05	Allow the player to inspect objects and receive clues.	Must
FR-06	Track health and prevent it from exceeding its maximum.	Must
FR-07	Allow the player to collect and use inventory items.	Must
FR-08	Provide turn-based combat with attack, item and flee options.	Should
FR-09	Track quest objectives and important story decisions.	Must
FR-10	Validate input and handle invalid commands without crashing.	Must
FR-11	Provide at least two different endings.	Should
FR-12	Save and load progress from a local file.	Could
FR-13	Allow the player to quit safely at any time outside a critical action.	Must

4. Non-Functional Requirements

- **Compatibility:** compile and run on supported Windows 10/11 systems.
- **Performance:** ordinary text interactions should respond immediately; no graphics hardware is required.
- **Reliability:** invalid input, missing save files and unexpected file contents should be handled safely.
- **Usability:** commands and available choices must be clearly displayed.

- **Maintainability:** separate game systems into focused classes and source files.
- **Portability:** use standard C++ features where possible; avoid unnecessary operating-system dependencies.
- **Data integrity:** clamp health and other bounded values; validate save data before using it.

Acceptance criteria

- The project compiles without errors using the documented build command.
- The generated EXE starts on a Windows machine with no compiler window or IDE required.
- The player can start a game, explore at least three locations and complete the main objective.
- Inventory, health, combat and at least one puzzle work correctly.
- Invalid input does not terminate the game unexpectedly.
- The game reaches a clear win or loss state and displays an ending message.

5. Libraries and Development Tools

For the first version, use the C++ standard library only. A text-based game does not need Raylib, SDL, OpenGL or Unreal Engine. Avoid adding dependencies until a feature genuinely requires one.

Library / tool	Use	Required?
iostream	Print text and read player input.	Yes
string	Store names, commands and descriptions.	Yes
vector	Store inventory items, choices and collections.	Yes
unordered_map	Look up locations, items or game flags by key.	Recommended
memory	Use smart pointers if dynamic ownership is needed.	Optional
fstream	Read and write save files.	Optional for v1
sstream	Parse command lines and file data.	Recommended
algorithm	Use helpers such as find, clamp and sort.	Recommended
random	Generate random encounters or outcomes.	Optional
limits	Safely handle invalid numeric input.	Recommended
CMake	Configure and build the project.	Recommended

Compiler and editor

- **Compiler:** MSYS2 UCRT64 MinGW-w64 GCC, or Visual Studio 2022 MSVC.
- **Language standard:** C++20.
- **Editor:** Visual Studio Code or Visual Studio.
- **Build system:** direct g++ command for a small first build; CMake as the project grows.
- **Version control:** Git and GitHub for source code, documentation and releases.

Important: the C++ standard library is included with a properly installed C++ toolchain. You do not download each standard header separately.

6. What Needs to Be Designed

System	Design decisions and responsibilities
Game flow	Title screen, new game, game loop, pause/quit behaviour, victory and defeat.
Player	Name, health, maximum health, inventory and current location.
Locations	Unique ID, title, description, exits, locked paths and points of interest.
Items	ID, name, description, type, effect and whether it can be consumed.
Inventory	Capacity (if any), adding/removing items, checking and using items.
Combat	Enemy health/damage, player actions, turn order, flee rules and rewards.
Quests	Objectives, prerequisites, completed flags and story decision flags.
Puzzles	Clues, accepted answers, attempts and unlock conditions.
Save system	Save format, file location, versioning and invalid-file handling.
Console UI	Consistent headings, menus, prompts, status display and input validation.

Suggested console screen

```
========================================
            THE LAST SIGNAL
========================================
Location: Greyhaven - Main Street

Rain falls over the empty street.
A weak light flickers inside the old shop.

Health:  80 / 100
Inventory: 2 items

What would you like to do?
1. Explore the old shop
2. Inspect the radio tower
3. Check inventory
4. View objectives
5. Quit game

Enter your choice:
```

User experience rules

- Show the available actions every time the player reaches a decision point.
- Use numbered menus for major choices and clear command names for optional actions.
- Tell the player why an action failed, such as a locked door or missing key.
- Keep story descriptions readable; avoid dumping several paragraphs after every action.
- Do not silently discard invalid input. Ask the player to try again.

7. Recommended C++ Project Structure

Keep declarations in header files (.h or .hpp) and function definitions in source files (.cpp). Each class should have one clear responsibility.

```
TheLastSignal/
|
|-- CMakeLists.txt
|-- README.md
|-- .gitignore
|
|-- include/
|   |-- Game.h
|   |-- Player.h
```

```
|   |-- Location.h
|   |-- Item.h
|   |-- Inventory.h
|   |-- Enemy.h
|   |-- CombatSystem.h
|   |-- QuestManager.h
|   |-- SaveSystem.h
|   |-- ConsoleUI.h
|
|-- src/
|   |-- main.cpp
|   |-- Game.cpp
|   |-- Player.cpp
|   |-- Location.cpp
|   |-- Item.cpp
|   |-- Inventory.cpp
|   |-- Enemy.cpp
|   |-- CombatSystem.cpp
|   |-- QuestManager.cpp
|   |-- SaveSystem.cpp
|   |-- ConsoleUI.cpp
|
|-- data/
|   |-- locations.txt
|   |-- items.txt
|   |-- enemies.txt
|
|-- saves/
|
|-- build/
|-- README.md
```

Class responsibilities

Class	Responsibility
Game	Owns the main game flow and coordinates systems.
Player	Stores player name, health, location and player state.
Location	Represents an explorable area and its exits.
Item	Represents an object the player can collect or use.
Inventory	Manages the player's collection of items.
Enemy	Stores enemy stats and behaviour data.
CombatSystem	Runs turn-based encounters and applies damage.
QuestManager	Tracks objectives and important story flags.
SaveSystem	Serialises and restores progress from a file.
ConsoleUI	Displays menus, prompts and formatted messages.

> Do not create every class immediately. Start with Game, Player and Location, then introduce additional classes when the matching feature is ready.

8. Basic C++ Program Structure

The following is a small starting point. It demonstrates the entry point, a game object, a loop and input handling. It is not the complete game; build the systems in stages.

```
#include <iostream>
#include <string>

class Game {
public:
    void run() {
```

```
        bool isRunning = true;

        std::cout << "Welcome to The Last Signal!\\n";

        while (isRunning) {
            std::cout << "\\n1. Explore\\n";
            std::cout << "2. View status\\n";
            std::cout << "3. Quit\\n";
            std::cout << "Choice: ";

            std::string choice;
            std::getline(std::cin, choice);

            if (choice == "1") {
                std::cout << "You explore the street...\\n";
            }
            else if (choice == "2") {
                std::cout << "Health: 100 / 100\\n";
            }
            else if (choice == "3") {
                isRunning = false;
            }
            else {
                std::cout << "Invalid choice. Try again.\\n";
            }
        }

        std::cout << "Thanks for playing!\\n";
    }
};

int main() {
    Game game;
    game.run();
    return 0;
}
```

Important implementation guidance

- Use std::getline for menu input so that input containing spaces can be read safely.
- Compare input against known commands and reject unsupported choices.
- Keep the game loop inside Game::run or a dedicated loop function.
- Keep player data in Player rather than scattering health variables throughout the project.
- Pass objects by reference when appropriate; avoid unnecessary copies.
- Use std::unique_ptr or std::shared_ptr only when ownership and lifetime requirements justify dynamic allocation.
- Use const where a function should not modify an object.

9. Building a Windows .exe

With MSYS2 UCRT64 MinGW-w64 installed, open the UCRT64 terminal in the project directory. The following command compiles the source files into a Windows executable:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 ^
    -I include src/*.cpp -o TheLastSignal.exe
```

The caret (^) is a Windows Command Prompt line-continuation character. In PowerShell, use a single line instead:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -I include src/*.cpp -o TheLastSignal.exe
```

For a tiny first project containing only main.cpp, use:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 main.cpp -o TheLastSignal.exe
```

Running the game

```
.\TheLastSignal.exe
```

The executable is the compiled application. A Windows user can double-click it or launch it from PowerShell/Command Prompt. If the program uses only standard C++ facilities and the runtime is available on the

target PC, no separate game engine is required.

Release checklist

- Build in Release mode with optimisation enabled.
- Test the EXE from a different folder, not only from the IDE.
- If using external data files, include them in the release folder and resolve paths reliably.
- If GCC runtime DLLs are not available on the target computer, distribute the required runtime DLLs alongside the EXE or use an appropriate static runtime option where licensing and toolchain rules permit.
- Do not distribute only the EXE if the game depends on external save templates, story files or other assets.
- Test on a clean Windows machine or virtual machine before publishing.

Suggested release folder

```
TheLastSignal-Release/
|-- TheLastSignal.exe
|-- data/
|-- README.txt
```

10. Development Plan and Testing

Phase	Build	Test before moving on
1. Foundation	Create project, main.cpp, Game class and title screen.	Program compiles, starts and exits cleanly.
2. Player	Add Player class, name and health.	Health values and status display are correct.
3. World	Add three connected locations and movement.	Only valid exits can be entered.
4. Inventory	Add items, collect, inspect and use.	Items are added, removed and applied correctly.
5. Combat	Add one enemy and turn-based combat.	Damage, healing, defeat and victory work.
6. Story	Add clues, one puzzle and quest flags.	Puzzle unlocks the correct route.
7. Endings	Add choices and multiple endings.	Different choices lead to expected outcomes.
8. Save/load	Add local save file support.	Progress restores correctly; invalid saves fail safely.
9. Release	Build and test standalone EXE.	Runs outside the development environment.

Minimum test cases

- Enter a valid menu choice and confirm the correct action occurs.
- Enter letters, blank input and unsupported numbers; the game should recover.
- Attempt to move through a locked exit without its key.
- Collect the same item twice and confirm the intended duplicate rule.
- Use a healing item at maximum health and verify health does not exceed the maximum.
- Reduce player health to zero and verify the defeat flow.
- Defeat an enemy and verify rewards or progression are applied once.
- Load a missing or malformed save file and show a safe error message.
- Complete each ending path and verify its final message.

11. Final Deliverables

- **Source code:** all .h/.hpp and .cpp files.
- **Executable:** TheLastSignal.exe built for Windows.
- **README.md:** game overview, controls, requirements, build and run instructions.
- **Game documentation:** storyline, requirements, architecture and design decisions.
- **Test checklist:** test cases and results.
- **Release folder:** EXE plus any required data files and runtime dependencies.
- **GitHub repository:** clean folder structure, meaningful commits and a clear project README.

Suggested GitHub repository description

A C++20 text-based adventure RPG for Windows. Explore the abandoned settlement of Greyhaven, solve puzzles, manage your inventory and uncover the mystery behind a signal that should never have been transmitted.

Recommended first milestone

Begin with Phase 1 only: create the project folder, compile a simple main.cpp, display the title screen and implement a working New Game / Quit menu. Test that the EXE runs successfully before adding player systems or story content.

This staged approach keeps the project manageable and makes it easier to identify and fix problems as your C++ skills develop.